

[WS03] Prepare Server

308-493 CROSS-PLATFORM MOBILE PROGRAMMING AND APPLICATIONS

สิ่งที่จะได้เรียนจาก workshop นี้

การใช้ Node.js เป็น Web Service

การใช้ Node.js เพื่อติดต่อกับระบบฐานข้อมูล MariaDB

การใช้ NPM เพื่อ install ตัว package ที่จำเป็น

- การใช้ Express เพื่อเพิ่ม Endpoint
- การใช้ Body-parser เพื่ออ่านและส่งข้อมูลแบบ JSON

การใช้ REST client สำหรับทดสอบ API

[WS03-1] การสร้าง Server ด้วย Node.js

การสร้าง Server ด้วย Node.js

Download โปรแกรม Node.js จาก <https://nodejs.org/en/download>
ติดตั้งโปรแกรม Node.js

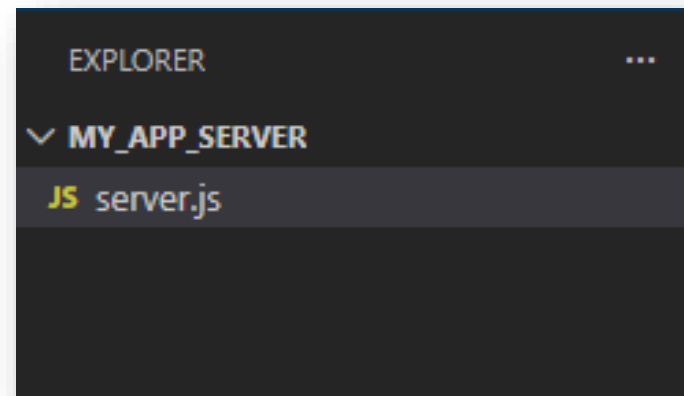
สร้าง Web server ด้วย Node.js

สร้าง folder ชื่อ
"my_app_server"

สร้าง file ชื่อ server.js

เปิด folder ชื่อ my_app_server
ด้วย VSCode

แก้ไขไฟล์ server.js ใน VSCode

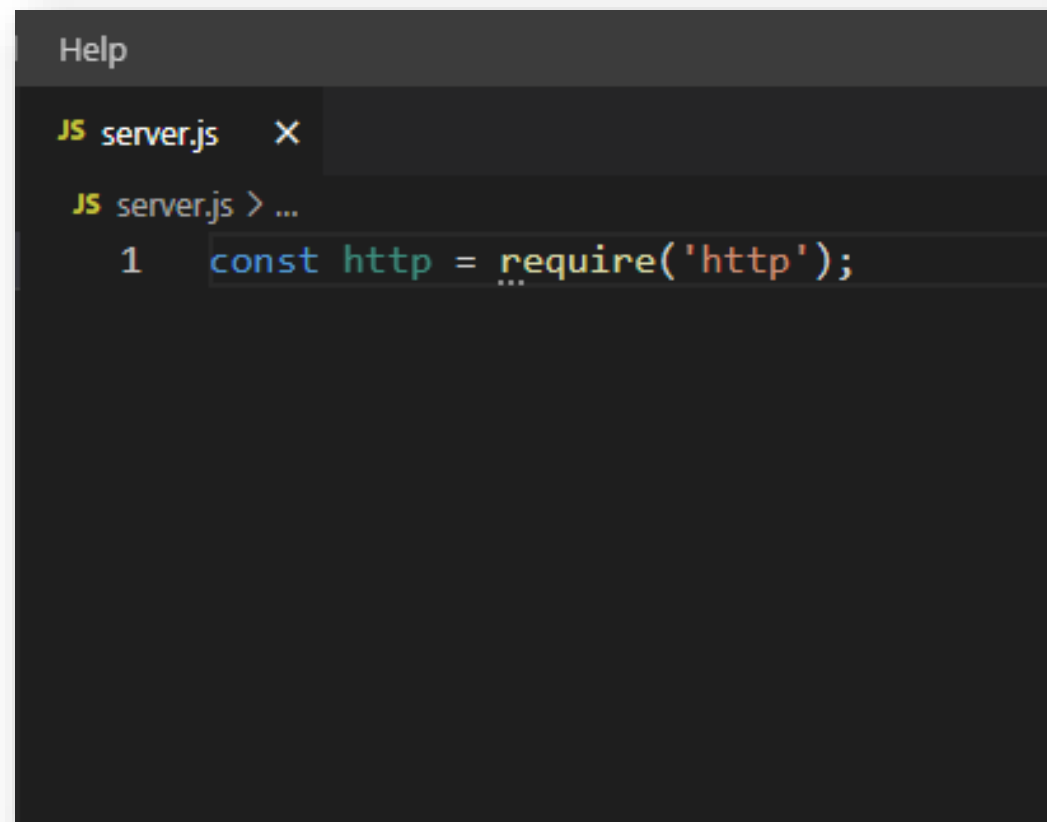


สร้าง Web server ด้วย Node.js

เรียกใช้โมดูลชื่อ **http**

โมดูล http เป็น library ที่มากับ node.js อยู่แล้ว

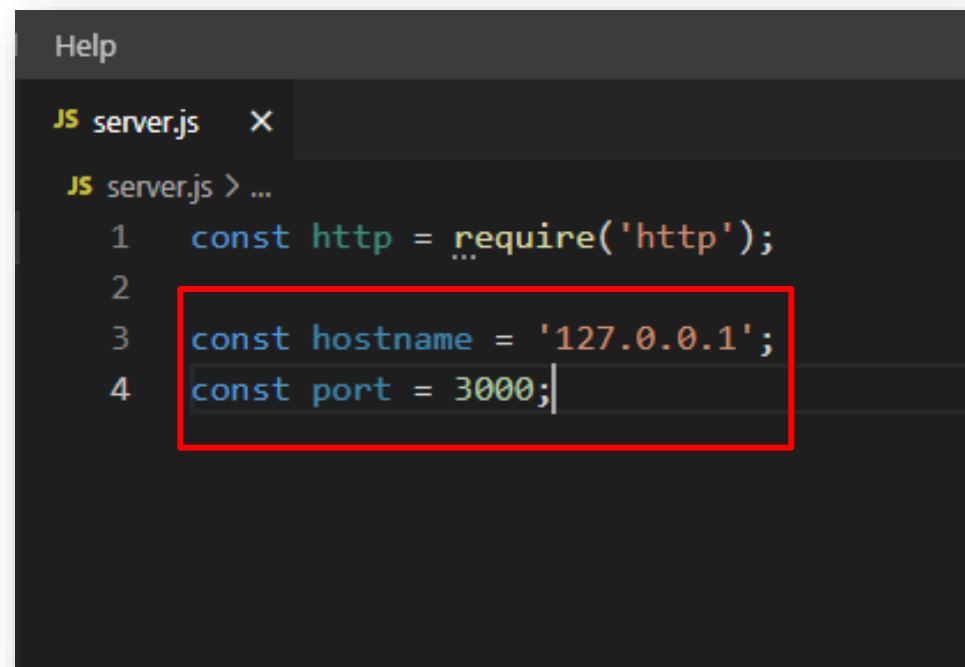
ใช้สำหรับทำให้เครื่องคอมพิวเตอร์ของเราเป็น Web server



```
Help
JS server.js  X
JS server.js > ...
1  const http = require('http');
```

สร้าง Web server ด้วย Node.js

กำหนด IP ของเครื่องและ
หมายเลข port สำหรับการ
เชื่อมต่อ

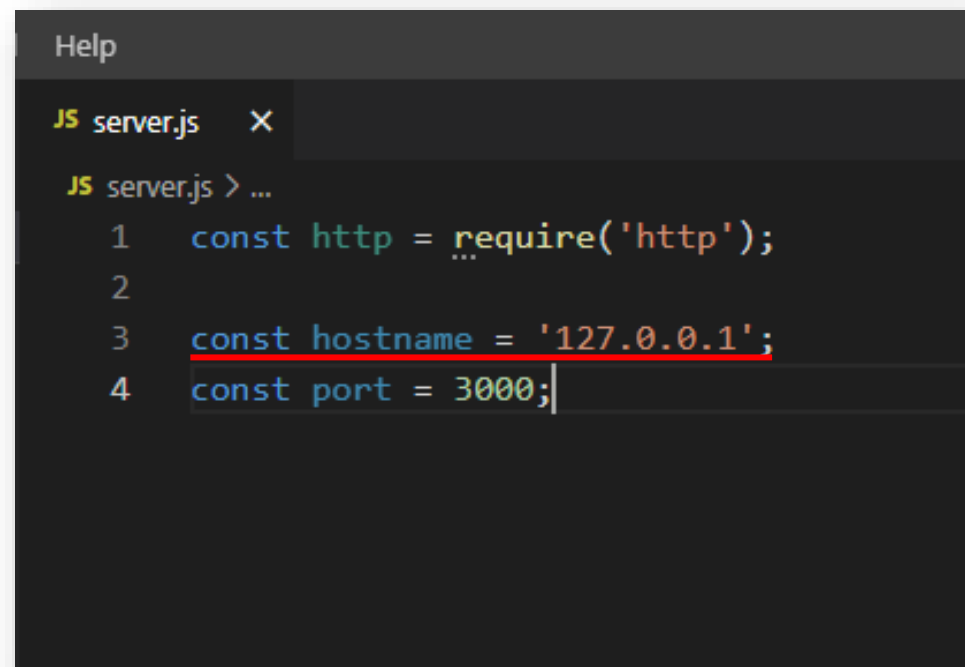


```
Help
JS server.js X
JS server.js > ...
1  const http = require('http');
2
3  const hostname = '127.0.0.1';
4  const port = 3000;
```

สร้าง Web server ด้วย Node.js

กำหนด IP Address สำหรับ
เครื่อง Server ที่เรากำลังสร้าง

ในกรณีนี้เราใช้ 127.0.0.1 ซึ่งมี
ความหมายเดียวกับ localhost
หรือ server ภายในเครื่องของเรา
นั่นเอง

A screenshot of a code editor window with a dark theme. The title bar at the top says 'Help'. Below it, there's a tab labeled 'JS server.js' with a close button 'X'. The editor shows a command prompt 'JS server.js > ...' followed by four lines of JavaScript code. Line 1: 'const http = require('http');'. Line 2: an empty line. Line 3: 'const hostname = '127.0.0.1';' where the string '127.0.0.1' is underlined in red. Line 4: 'const port = 3000;'.

```
Help

JS server.js  X

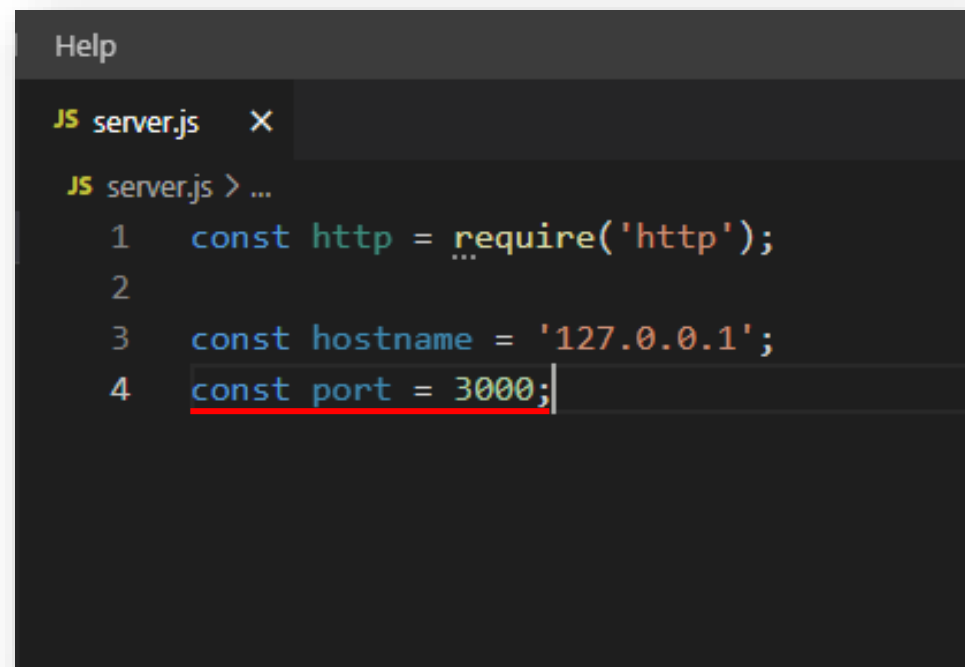
JS server.js > ...
1  const http = require('http');
2
3  const hostname = '127.0.0.1';
4  const port = 3000;
```

สร้าง Web server ด้วย Node.js

กำหนด port ให้กับ web server

การกำหนด port เป็นการเปิดช่องทางในการติดต่อกับเครื่องคอมพิวเตอร์อื่น ๆ

เครื่องอื่น ๆ จะติดต่อ web server ของเราผ่านเลข port ที่กำหนด

A screenshot of a code editor window with a dark theme. The title bar at the top says 'Help'. There are two tabs: 'JS server.js' and 'X'. The active tab 'JS server.js' shows a command prompt 'JS server.js > ...' followed by four lines of JavaScript code. Line 1: 'const http = require('http');'. Line 2: an empty line. Line 3: 'const hostname = '127.0.0.1';'. Line 4: 'const port = 3000;'. The text 'const port = 3000;' on line 4 is underlined with a red line. The cursor is at the end of line 4.

```
Help
JS server.js  X
JS server.js > ...
1  const http = require('http');
2
3  const hostname = '127.0.0.1';
4  const port = 3000;
```

สร้าง Web server ด้วย Node.js

เขียนโปรแกรมเพื่อสร้าง Server
กำหนดให้ Server ส่งคำว่า
"Hello World" กลับไปยัง client
ที่ร้องขอข้อมูล

```
JS server.js
JS server.js > ...
1  const http = require('http');
2
3  const hostname = '127.0.0.1';
4  const port = 3000;
5
6  const server = http.createServer((req, res) => {
7    res.statusCode = 200;
8    res.setHeader('Content-Type', 'text/plain');
9    res.end('Hello World');
10 });
```

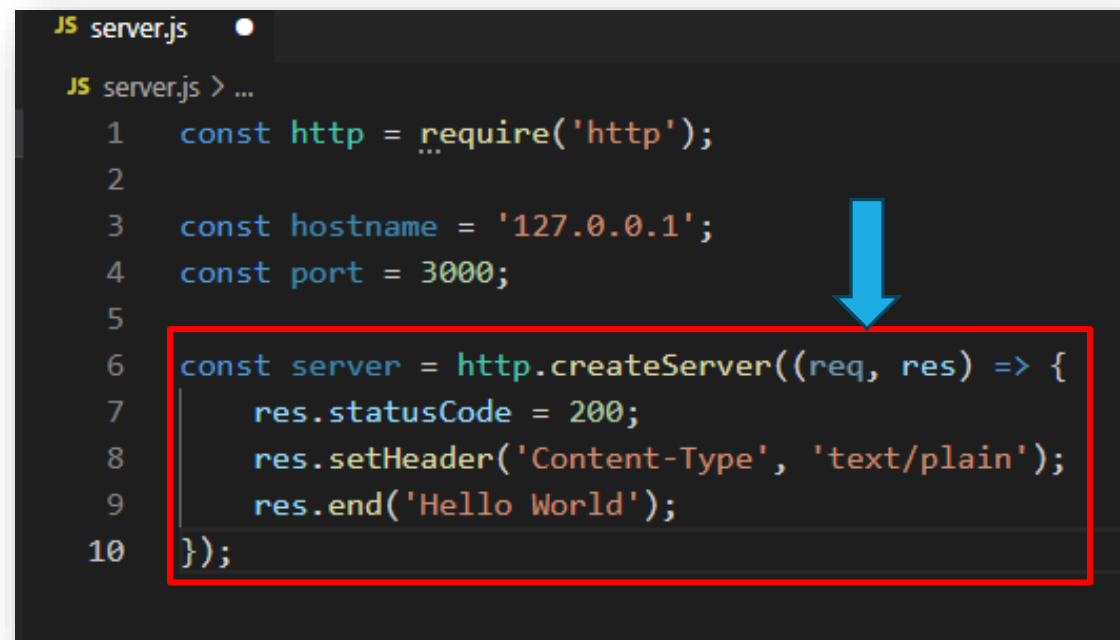
สร้าง Web server ด้วย Node.js

สังเกต (req,res) => {...}

รูปแบบนี้เป็น anonymous function สำหรับใช้เป็น callback ซึ่งจะทำงานเมื่อเกิด event ที่เข้าเงื่อนไข

เงื่อนไขในกรณีนี้ คือเมื่อผู้ใช้พิมพ์ URL เป็น localhost:3000

ตัวแปร req (request) และ res (response) คือตัวแปรที่ node ส่งกลับมาให้เราใช้ใน callback



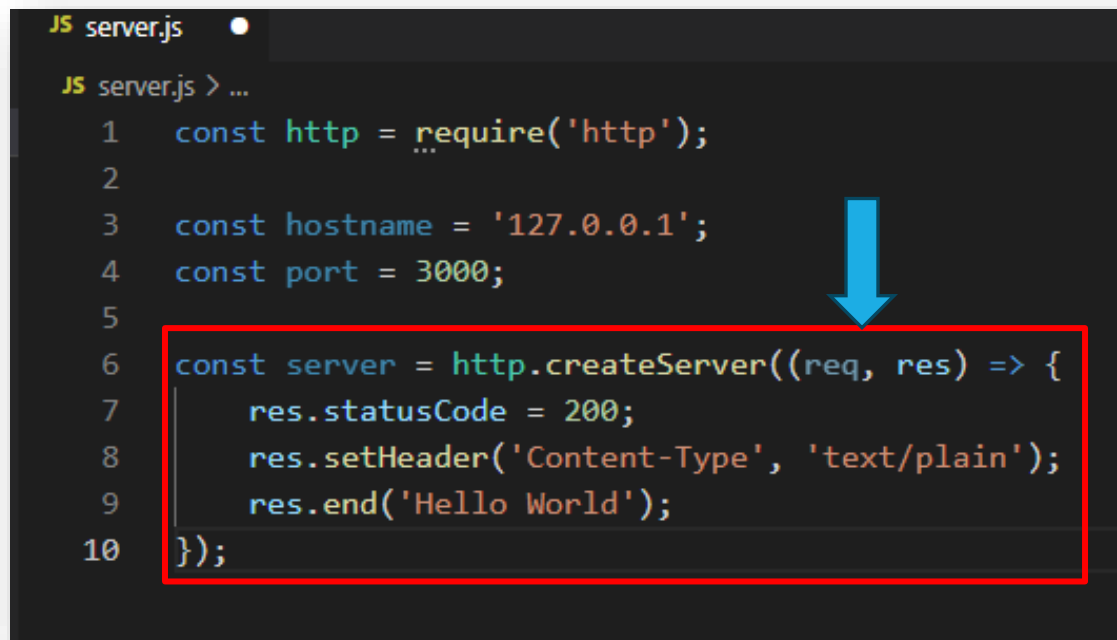
```
JS server.js
JS server.js > ...
1  const http = require('http');
2
3  const hostname = '127.0.0.1';
4  const port = 3000;
5
6  const server = http.createServer((req, res) => {
7    res.statusCode = 200;
8    res.setHeader('Content-Type', 'text/plain');
9    res.end('Hello World');
10 });
```

Request => ผู้ใช้ขอ
Response => server ส่ง

สร้าง Web server ด้วย Node.js

สังเกต (req,res) => {...}

เป็นรูปแบบย่อของ function(req, res) => {...}



```
JS server.js
JS server.js > ...
1  const http = require('http');
2
3  const hostname = '127.0.0.1';
4  const port = 3000;
5
6  const server = http.createServer((req, res) => {
7    res.statusCode = 200;
8    res.setHeader('Content-Type', 'text/plain');
9    res.end('Hello World');
10 });
```

A blue arrow points to the callback function in the `http.createServer` call, which is also enclosed in a red box.

สร้าง Web server ด้วย Node.js

เปิดการทำงานของ server

กำหนดให้ server ตอบสนอง
client ผ่าน IP address และ port
ที่เรากำหนด

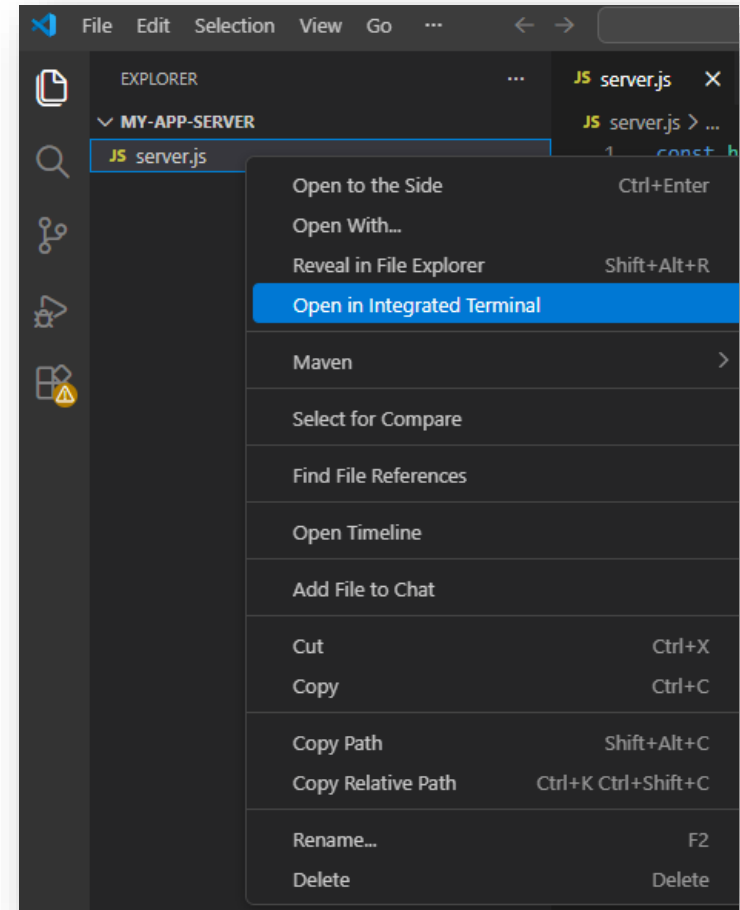
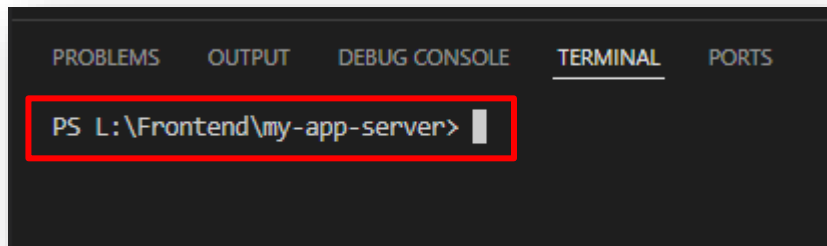
```
JS server.js
JS server.js > ...
1  const http = require('http');
2
3  const hostname = '127.0.0.1';
4  const port = 3000;
5
6  const server = http.createServer((req, res) => {
7    res.statusCode = 200;
8    res.setHeader('Content-Type', 'text/plain');
9    res.end('Hello World');
10 });
11
12 server.listen(port, hostname, () => {
13   console.log(`Server running at http://${hostname}:${port}/`);
14 });
```

ทดสอบ Web server

เปิด Terminal โดยคลิกขวาที่ไฟล์
server.js แล้วเลือก Open in
Integrated Terminal

จะปรากฏหน้าต่าง Terminal ที่
ด้านล่างของหน้าจอ

ตรวจสอบว่า folder ตรงกับ
folder ของเราหรือไม่



ทดสอบ Web server

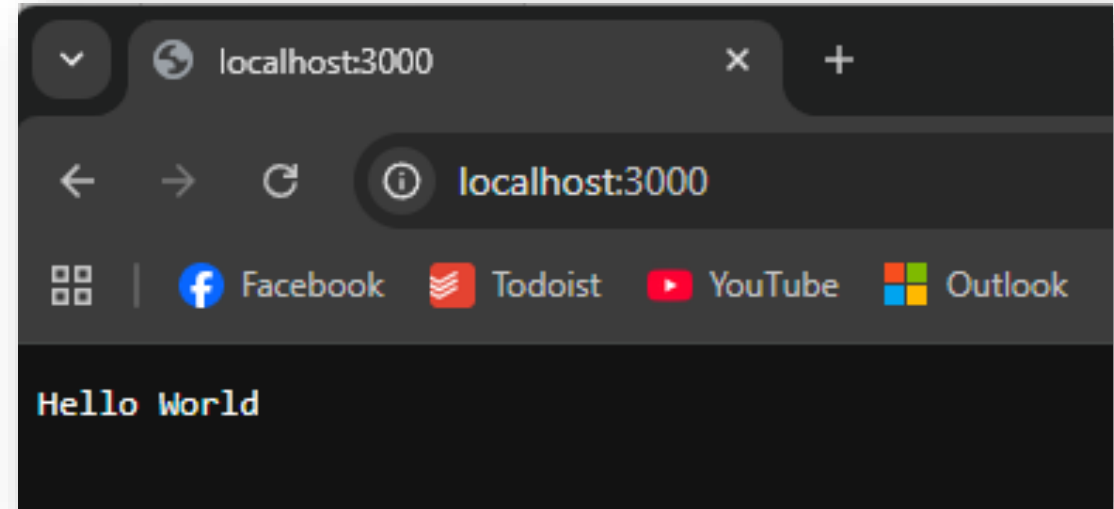
พิมพ์คำสั่ง `node server.js` ลงใน Terminal

เปิด Web browser แล้วพิมพ์

`127.0.0.1:3000` (หรือ `localhost:3000`) ลงในช่อง URL

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS L:\Frontend\my-app-server> node server.js
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS L:\Frontend\my-app-server> node server.js
Server running at http://127.0.0.1:3000
```



ทดลองแก้ไข Web server

เปลี่ยนคำว่า Hello World เป็นคำว่า Hello Uncle

ในหน้าต่าง Terminal ให้นักศึกษากดปุ่ม Ctrl + C เพื่อหยุดการทำงานของ server

สั่งให้ server ทำงานและทดสอบอีกครั้ง

ทดลองใส่ localhost:3000/user ในช่อง URL แล้วสังเกตผล หากเหตุผลทำให้เกิดเหตุการณ์ดังกล่าว

[WS03-2] ติดตั้ง Package เสริมให้กับ Server

NPM

Node Package Manager

เป็นโปรแกรมสำหรับการจัดการชุดคำสั่งภายนอกที่ผู้พัฒนา application ต้องการนำมาใช้ในโปรแกรม (เราเรียกว่า Package) เช่น

- mysql2 สำหรับการติดต่อกับฐานข้อมูล
- express สำหรับการจัด route เพื่อให้ server ของเราสามารถรองรับ URI ได้หลายแบบ
 - URI คือตำแหน่งของเว็บที่เราต้องการให้ Web browser เปิด

Package ต่าง ๆ ที่เราสามารถใช้นpm ได้เป็น open source

- สามารถนำมาใช้ในโปรแกรมของเราได้โดยไม่เสียค่าใช้จ่าย

โปรแกรม npm ถูกติดตั้งมาพร้อมกับ node.js

NPM

npm กับโปรเจกต์ของเรา

จาก Workshop 1 โปรแกรมของเรามีเพียงแค่ตัว server (server.js) เท่านั้น

หากเราต้องการใช้ package ต่าง ๆ จาก npm

- เราต้องติดตั้ง npm ให้กับตัวโปรเจกต์ของเรา

เปิด Terminal ในแฟ้ม my-app-server แล้วพิมพ์คำสั่ง

```
npm init
```

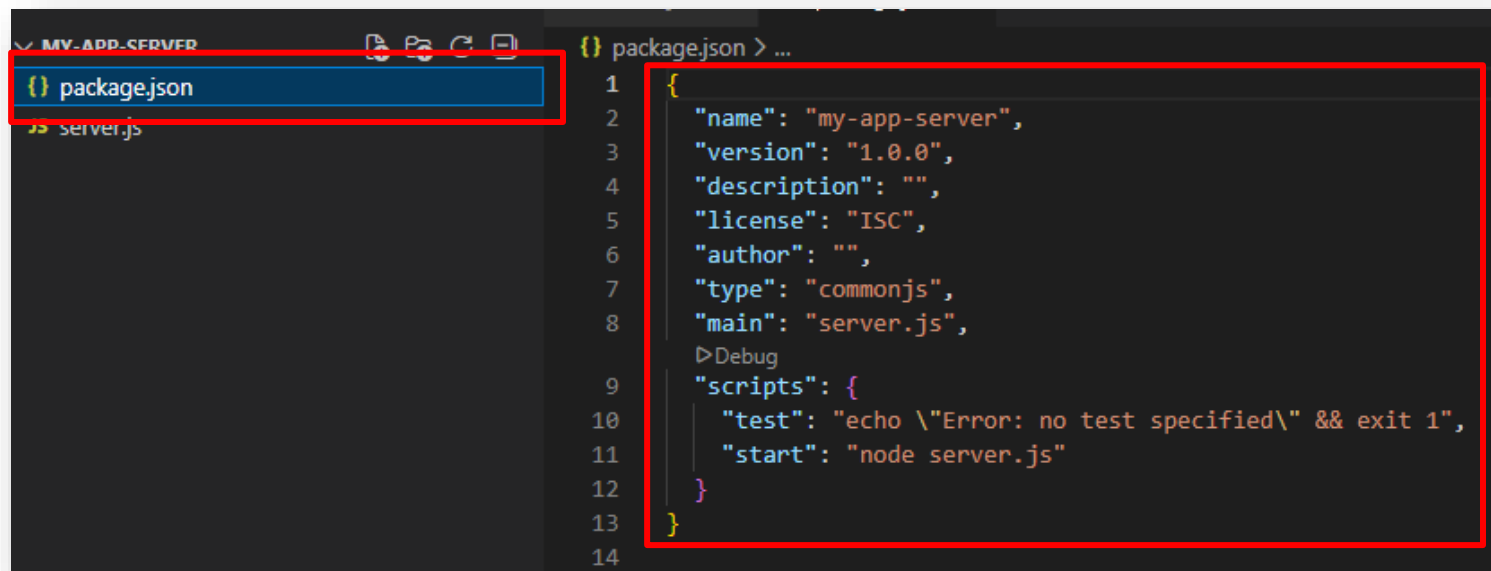
NPM

npm กับโปรเจกต์ของเรา

เมื่อเราเพิ่มข้อมูลต่าง ๆ แล้ว ตัว npm จะสร้างไฟล์ขึ้นมา 1 ไฟล์ชื่อว่า **package.json**

ไฟล์นี้จะบันทึก

- ข้อมูลที่เราป้อนเข้าไปในขั้นตอน npm init
- Package ต่าง ๆ ที่เรานำมาใช้โดย npm



```
1 {  
2   "name": "my-app-server",  
3   "version": "1.0.0",  
4   "description": "",  
5   "license": "ISC",  
6   "author": "",  
7   "type": "commonjs",  
8   "main": "server.js",  
9   "scripts": {  
10     "test": "echo \"Error: no test specified\" && exit 1",  
11     "start": "node server.js"  
12   }  
13 }  
14
```

การรับส่งข้อมูลในรูปแบบของ JSON

จาก WS03-1

- ไม่ว่าเราจะเปลี่ยน URL อย่างไร server.js ก็จะต้องสนองแบบเดิม
- เราจะใช้ Express เพื่อให้ server.js ตอบสนองตาม URL ที่ผู้ใช้ request เข้ามา

เมื่อเราจะอ่านและส่งข้อมูลในรูปแบบของ JSON

- เราจะใช้ border-parser เพื่อกำหนดให้เราส่งข้อมูลกลับเป็น JSON เสมอ

```
npm install express
```

```
npm install body-parser
```

ทดสอบการเรียกใช้ API

แก้ไขไฟล์ server.js ตามภาพประกอบ

```
const http = require('http');
const bp = require('body-parser');
const express = require('express');

const app = express();
app.use(bp.urlencoded({ extended: true }));
app.use(bp.json());

const hostname = '127.0.0.1';
const port = 3000;

app.get("/api/users", (req, res) => {
  var response = {
    isError: true,
    data: "You are unauthorized for this data"
  };

  res.send(JSON.stringify(response));
});

app.listen(port, () => {
  console.log(`Server running at http://${hostname}:${port}`);
});
```

ทดสอบการเรียกใช้ API

สร้าง endpoint เพื่อรองรับการร้องขอจาก client

Express จะส่งข้อมูล 2 ตัวมาให้ callback

- req → Request (รับเข้า)
- res → Response (ส่งคืน)

Client จะร้องขอด้วย URI

- <http://localhost:3000/api/users>

```
const http = require('http');
const bp = require('body-parser');
const express = require('express');

const app = express();
app.use(bp.urlencoded({ extended: true }));
app.use(bp.json());

const hostname = '127.0.0.1';
const port = 3000;

app.get("/api/users", (req, res) => {
  var response = {
    isError: true,
    data: "You are unauthorized for this data"
  };

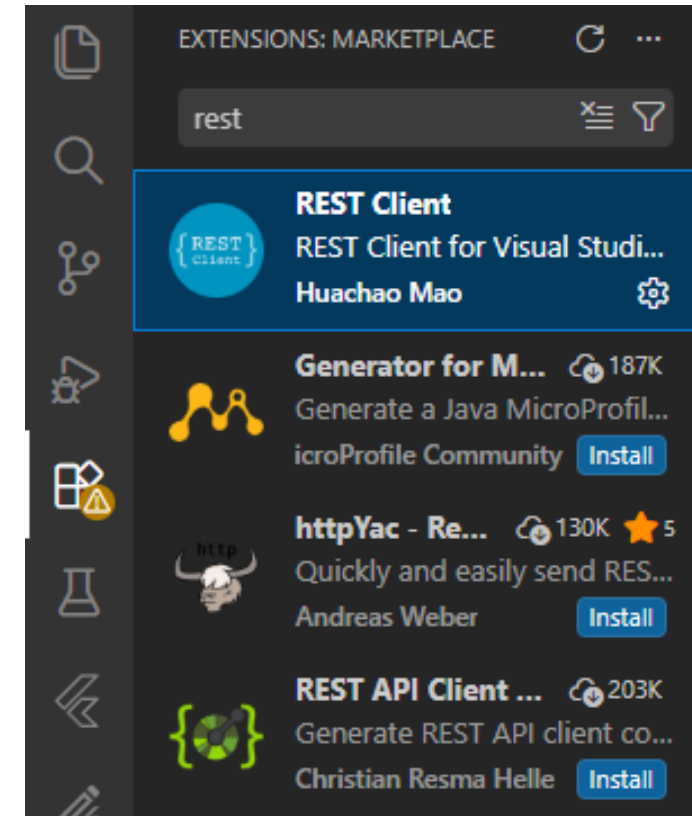
  res.send(JSON.stringify(response));
});

app.listen(port, () => {
  console.log(`Server running at http://${hostname}:${port}`);
});
```

ทดสอบการเรียกใช้ API

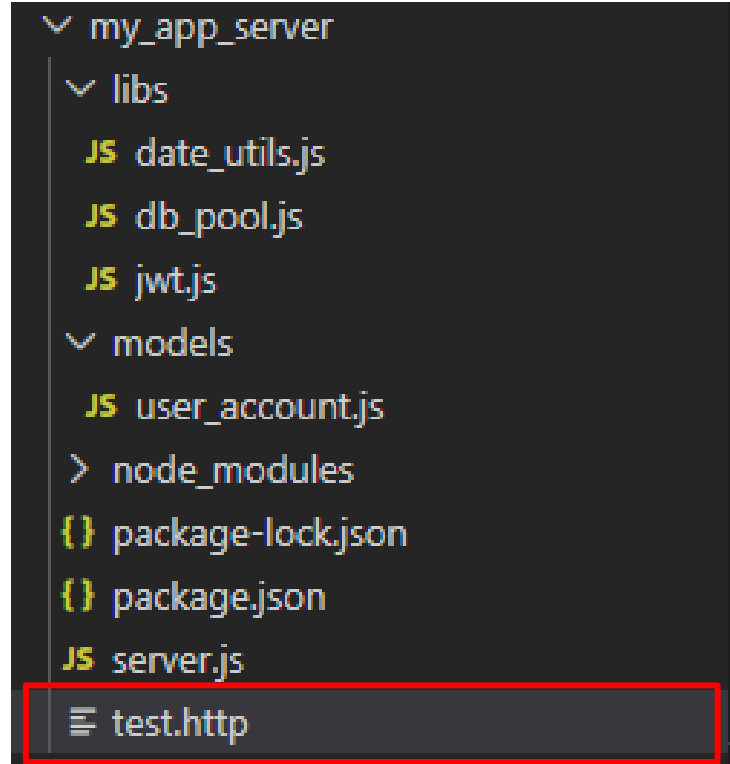
Install ส่วนเสริมของ VS Code ชื่อ REST Client

**หากมี postman อยู่แล้ว ไม่ต้อง install ตัวนี้ก็ได้อีก



ทดสอบการเรียกใช้ API

สร้างไฟล์ชื่อ test.http ใน my_app_server



```

my_app_server
├── libs
│   ├── JS date_utils.js
│   ├── JS db_pool.js
│   └── JS jwt.js
├── models
│   └── JS user_account.js
├── node_modules
├── {} package-lock.json
├── {} package.json
├── JS server.js
└── test.http

```

ทดสอบการเรียกใช้ API

แก้ไขไฟล์ test.http ตามภาพประกอบ
กดที่ปุ่ม Send Request

```
Send Request
GET http://localhost:3000/api/users HTTP/1.1
Content-Type: application/json

{

}
```

```
Response(3ms) X

1 HTTP/1.1 200 OK
2 X-Powered-By: Express
3 Access-Control-Allow-Origin: *
4 Content-Type: text/html; charset=utf-8
5 Content-Length: 60
6 ETag: W/"3c-XdgHDIjHrx19pf4RcHeqETMFLEQ"
7 Date: Wed, 01 Jul 2026 03:32:23 GMT
8 Connection: close
9
10 {
11   "isError": true,
12   "data": "You are unauthorized for this data"
13 }
```

ทดสอบการเรียกใช้ API

เพิ่ม endpoint ใหม่ ตามภาพประกอบ

Endpoint ตัวนี้จะรับข้อมูลจาก client ผ่าน method แบบ POST

ข้อมูลที่ client ส่งมาจะอยู่ในส่วนของ body ของ request

```
app.get("/api/users", (req, res) => {  
  var response = {  
    isError: true,  
    data: "You are unauthorized for this data"  
  };  
  
  res.send(JSON.stringify(response));  
});
```

```
app.post("/api/multiple_by_2", (req, res) => {  
  var response = {  
    isError: false,  
    data: {  
      no1: req.body.no_1 * 2,  
      no2: req.body.no_2 * 2  
    }  
  };  
  
  res.send(JSON.stringify(response));  
});
```

ทดสอบการเรียกใช้ API

แก้ไข test.http เพื่อเรียกใช้ API ตัวใหม่

```
Send Request
GET http://localhost:3000/api/users HTTP/1.1
Content-Type: application/json

{
}

###
Send Request
POST http://localhost:3000/api/multiple_by_2 HTTP/1.1
Content-Type: application/json

{
  "no_1": 20,
  "no_2": 50.5
}
```

```
HTTP/1.1 200 OK
X-Powered-By: Express
Content-Type: text/html; charset=utf-8
Content-Length: 45
ETag: W/"2d-ReMf/aIXug8vKZVTUZGL2NcsS8Y"
Date: Wed, 01 Jul 2026 03:39:05 GMT
Connection: close

✓{
  "isError": false,
  "data": {
    "no1": 40,
    "no2": 101
  }
}
```

MariaDB

Download และติดตั้ง MariaDB จาก <https://mariadb.org/>

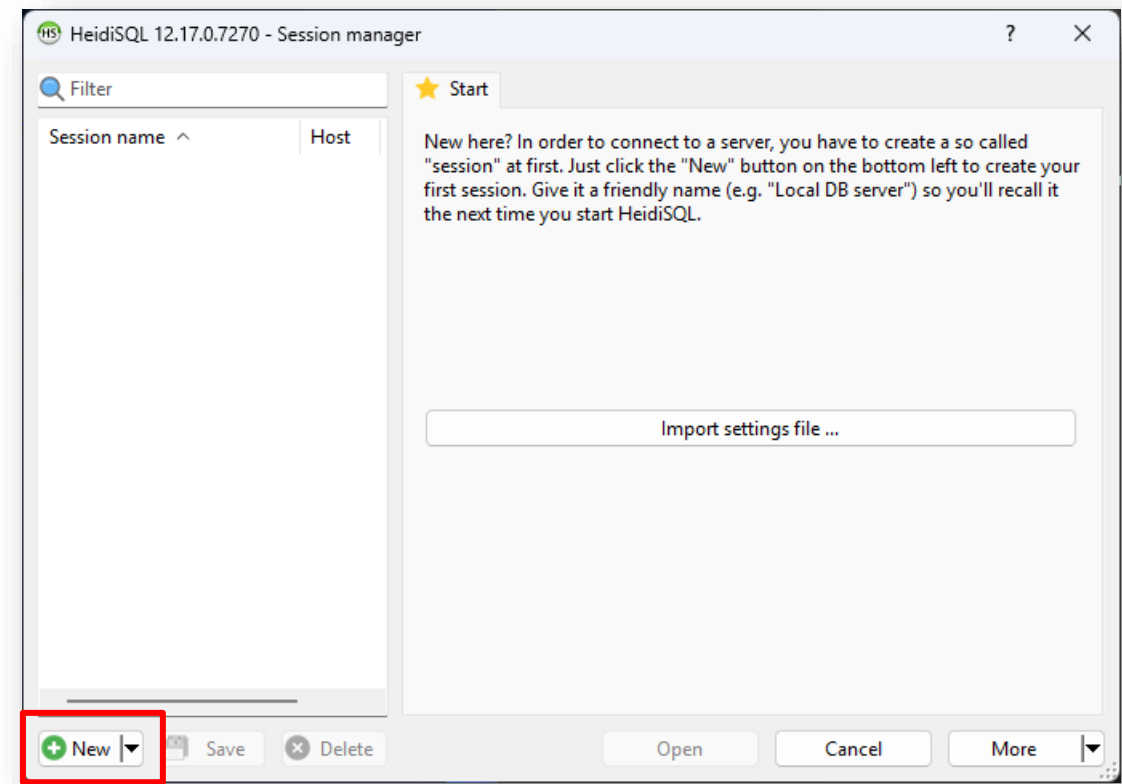
Installer จะติดตั้ง MariaDB พร้อมกับ HeidiSQL ซึ่งเป็น front-end สำหรับใช้งานกับฐานข้อมูลมาให้

ในกรณีที่เคยลง XAMPP ไว้

- ให้กำหนด port ของ MariaDB เป็นตัวเลขอื่น เช่น 3307

MariaDB

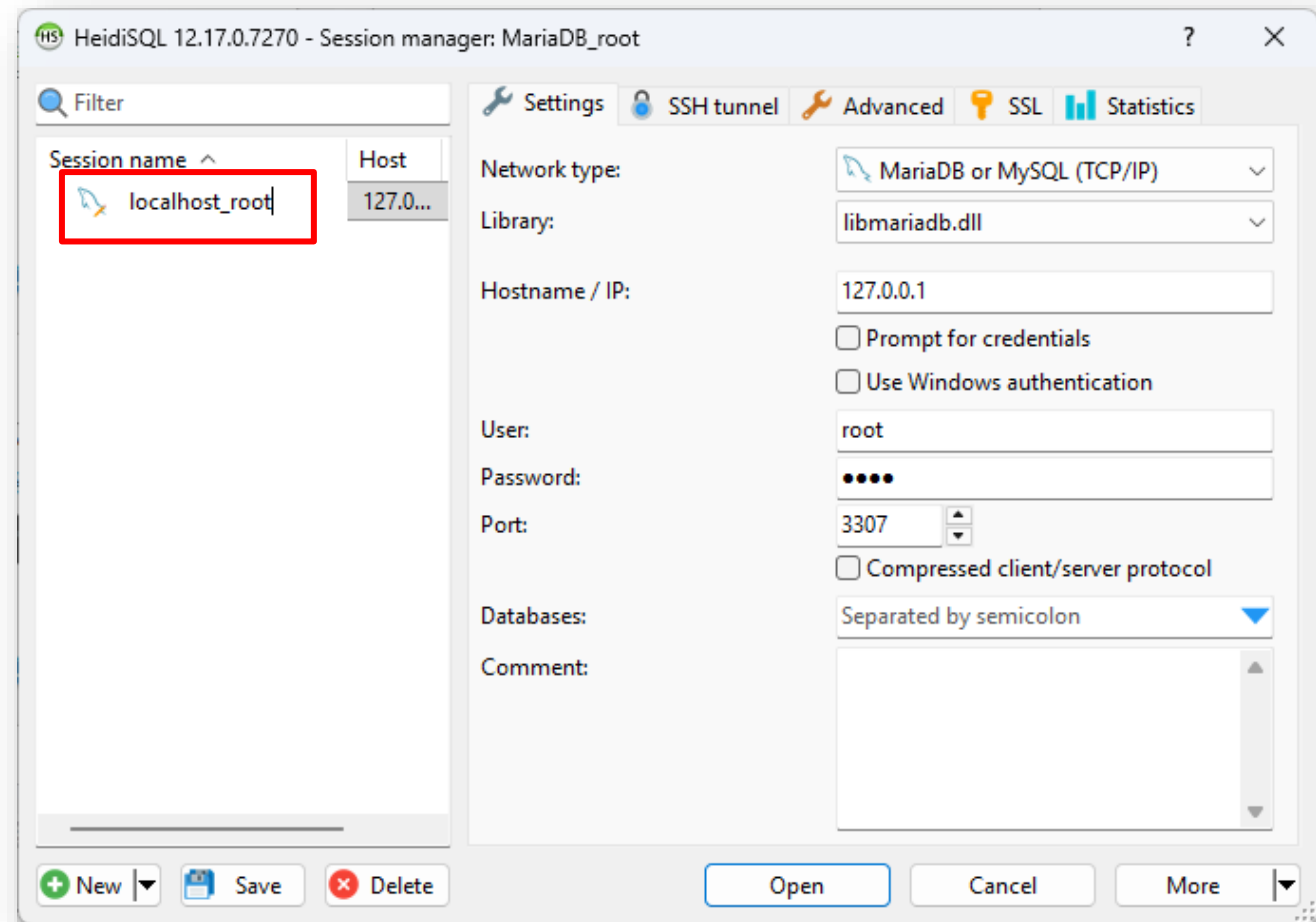
สร้าง connection ใหม่โดยกดที่ปุ่ม
New



MariaDB

ตั้งชื่อ connection ว่า
localhost_root

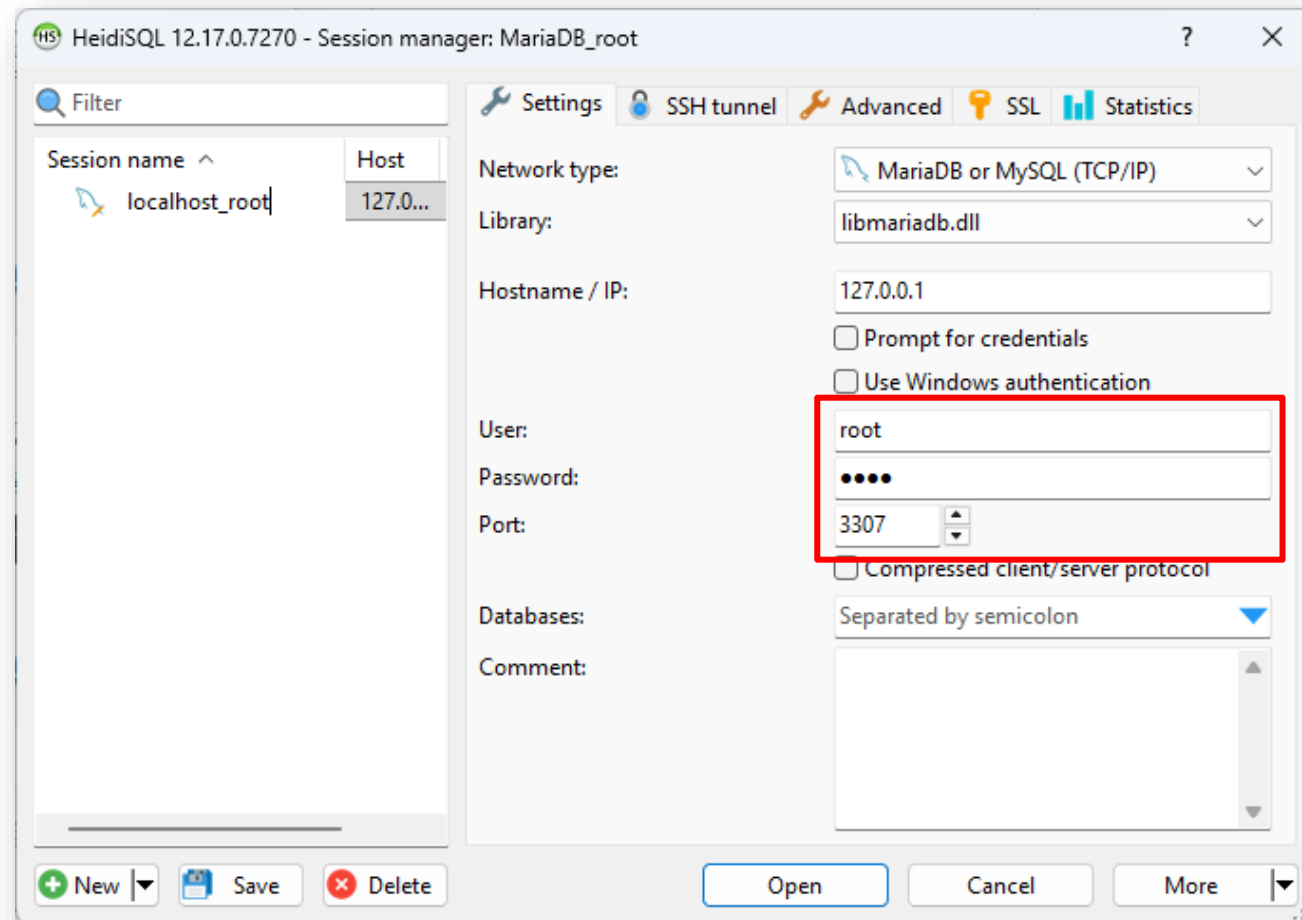
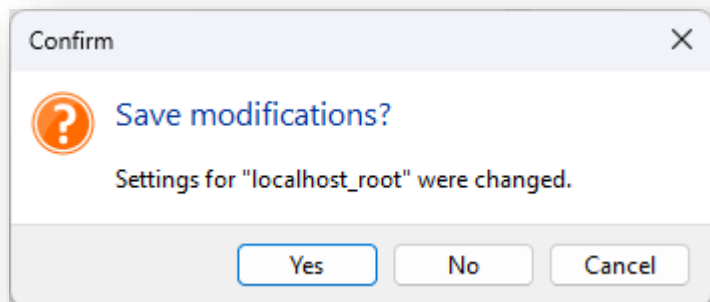
เพื่อเตือนความจำว่า db
อยู่ในเครื่องของเราและเรา
กำลังจะเข้าใช้ด้วย
username: root



MariaDB

ป้อน username password และเลข port ตามที่เราได้ตั้งไว้

กดปุ่ม Yes เพื่อยืนยันในการบันทึก connection



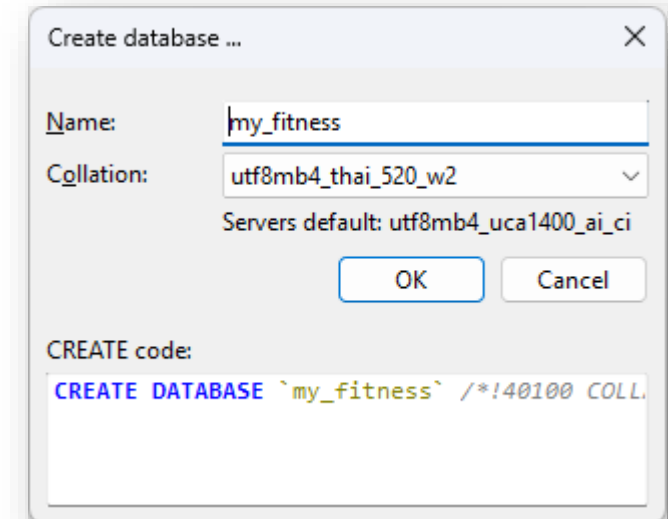
MariaDB

สร้างฐานข้อมูลใหม่ ชื่อ my_fitness

เลือก Collation เป็น utf8mb4_thai_520_w2

******ในกรณีที่เรามีข้อมูลเป็นภาษาไทย การตั้ง Collation ตัวอื่นอาจทำให้

- ข้อมูลภาษาไทยกลายเป็นภาษาต่างดาว
- เกิดความผิดพลาดในการ sorting



MariaDB

เปิดไฟล์ my_fitness.sql โดย
ลากไฟล์วางใน tab ชื่อ Query

Run คำสั่งโดยกดปุ่ม F9

ระบบจะสร้างตารางใหม่ให้



```
1  -- Host: 127.0.0.1
2  -- Server version: 12.3.2-MariaDB - MariaDB Server
3  -- Server OS: Win64
4  -- HeidiSQL Version: 12.17.0.7270
5
6  -----
7
8  /*!40101 SET @OLD_CHARACTER_SET_CLIENT=@@CHARACTER_SET_CLIENT */;
9  /*!40101 SET NAMES utf8 */;
10 /*!50503 SET NAMES utf8mb4 */;
11 /*!40103 SET @OLD_TIME_ZONE=@@TIME_ZONE */;
12 /*!40103 SET TIME_ZONE='+00:00' */;
13 /*!40014 SET @OLD_FOREIGN_KEY_CHECKS=@@FOREIGN_KEY_CHECKS, FOREIGN_KEY_CHECKS=0 */;
14 /*!40101 SET @OLD_SQL_MODE=@@SQL_MODE, SQL_MODE='NO_AUTO_VALUE_ON_ZERO' */;
15 /*!40111 SET @OLD_SQL_NOTES=@@SQL_NOTES, SQL_NOTES=0 */;
16
17
18 -- Dumping database structure for my_fitness
19 DROP DATABASE IF EXISTS `my_fitness`;
20 CREATE DATABASE IF NOT EXISTS `my_fitness` /*!40100 DEFAULT CHARACTER SET utf8mb4 COLLATE utf8mb4_th
21 USE `my_fitness`;
22
23 -- Dumping structure for table my_fitness.activities
24 DROP TABLE IF EXISTS `activities`;
25 CREATE TABLE IF NOT EXISTS `activities` (
26   `act_id` int(11) NOT NULL AUTO_INCREMENT,
27   `ex_id` int(11) NOT NULL,
28   `act_date` datetime NOT NULL DEFAULT current_timestamp(),
29   `kcal_expense` int(11) NOT NULL DEFAULT 0,
30   `account_id` int(11) NOT NULL,
31   PRIMARY KEY (`act_id`)
32 ) ENGINE=InnoDB AUTO_INCREMENT=4 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_thai_520_w2;
33
34 -- Dumping data for table my_fitness.activities: ~3 rows (approximately)
35 REPLACE INTO `activities` (`act_id`, `ex_id`, `act_date`, `kcal_expense`, `account_id`) VALUES
36 (1, 1, '2026-07-01 08:37:28', 200, 1);
```

MariaDB

พิมพ์คำสั่งใน Terminal ของ my_app_server เพื่อ install ตัว package ของ mariadb

```
npm install mariadb
```

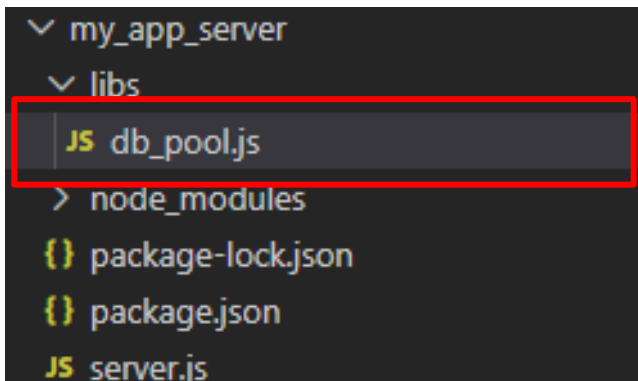
```
PS L:\Teaching Flutter\my_app_server> npm install mariadb
```

MariaDB

สร้าง folder ชื่อ libs

สร้างไฟล์ใหม่ชื่อ db_pool.js ใน libs

แก้ไขไฟล์ db_pool.js ตามภาพประกอบ



```
const mariadb = require('mariadb');
const pool = mariadb.createPool({
  host: 'localhost',
  user: 'root',
  password: '1234',
  port: 3307,
  connectionLimit: 5
});

module.exports = pool;
```

MariaDB

เราสร้างไฟล์ db_pool.js แยกออกมาเพื่อจะนำไปใช้ในอีกหลายไฟล์ที่เราจำเป็นต้องติดต่อกับฐานข้อมูล

Pool คือ connection ของ MariaDB ที่สร้างไว้เพื่อหมุนเวียนให้บริการ request ที่อาจเข้ามาพร้อมกันหลายตัว

```
const mariadb = require('mariadb');
const pool = mariadb.createPool({
  host: 'localhost',
  user: 'root',
  password: '1234',
  port: 3307,
  connectionLimit: 5
});

module.exports = pool;
```

MariaDB

การใช้ `module.exports` จะทำให้เราสามารถเรียกใช้ object (ในที่นี้คือ `pool`) จากภายนอกได้

ทำให้เราไม่ต้องสร้าง `pool` ซ้ำ ๆ

```
const mariadb = require('mariadb');
const pool = mariadb.createPool({
  host: 'localhost',
  user: 'root',
  password: '1234',
  port: 3307,
  connectionLimit: 5
});

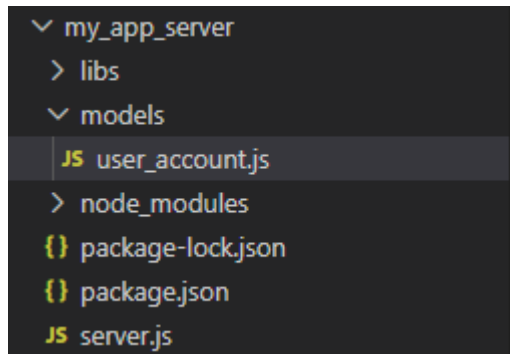
module.exports = pool;
```

MariaDB

สร้าง folder ชื่อ models

สร้างไฟล์ชื่อ user_account.js

แก้ไข user_account.js ตาม
ภาพประกอบ



```
const pool = require('../libs/db_pool');

module.exports = {
  getUserAccountById: async (accountId) => {
    let conn;
    let result;

    try {
      conn = await pool.getConnection();

      var sql = "SELECT account_id, account_username, account_image_url FROM user_accounts "
        + "WHERE account_id = ?";

      var rows = await conn.query(sql, [accountId]);

      result = {
        isError: false,
        data: rows
      };
    } catch (error) {
      result = {
        isError: false,
        errorMessage: error.message
      }
    } finally {
      if (conn)
        conn.release();
    }

    return result;
  }
}
```

MariaDB

ນຳ pool ໃນ db_pool.js ມາໃຊ້
ງານ

```
const pool = require('../libs/db_pool');

module.exports = {
  getUserAccountById: async (accountId) => {
    let conn;
    let result;

    try {
      conn = await pool.getConnection();

      var sql = "SELECT account_id, account_username, account_image_url FROM user_accounts "
        + "WHERE account_id = ?";

      var rows = await conn.query(sql, [accountId]);

      result = {
        isError: false,
        data: rows
      };
    } catch (error) {
      result = {
        isError: false,
        errorMessage: error.message
      };
    } finally {
      if (conn)
        conn.release();

      return result;
    }
  }
}
```

MariaDB

สร้าง function ชื่อ
getUserAccountById

```
const pool = require('../libs/db_pool');

module.exports = {
  getUserAccountById: async (accountId) => {
    let conn;
    let result;

    try {
      conn = await pool.getConnection();

      var sql = "SELECT account_id, account_username, account_image_url FROM user_accounts "
        + "WHERE account_id = ?";

      var rows = await conn.query(sql, [accountId]);

      result = {
        isError: false,
        data: rows
      };
    } catch (error) {
      result = {
        isError: false,
        errorMessage: error.message
      };
    } finally {
      if (conn)
        conn.release();

      return result;
    }
  }
}
```

MariaDB

สร้างตัวแปร

- conn สำหรับเก็บ connection ไปยัง MariaDB
- result สำหรับส่งคืนผลลัพธ์ การสืบค้นข้อมูลจาก MariaDB

```
const pool = require('../libs/db_pool');

module.exports = {
  getUserAccountById: async (accountId) => {
    let conn;
    let result;

    try {
      conn = await pool.getConnection();

      var sql = "SELECT account_id, account_username, account_image_url FROM user_accounts "
        + "WHERE account_id = ?";

      var rows = await conn.query(sql, [accountId]);

      result = {
        isError: false,
        data: rows
      };
    } catch (error) {
      result = {
        isError: false,
        errorMessage: error.message
      }
    } finally {
      if (conn)
        conn.release();

      return result;
    }
  }
}
```

MariaDB

สร้าง Try...Catch...Finally Block

โปรแกรมจะพยายาม run ใน ส่วน try ก่อน

หากเกิด error จะกระโดดไปที่ catch

เมื่อสิ้นสุดคำสั่งใน try หรือ catch แล้วจะมา run คำสั่งใน finally

```
const pool = require('../libs/db_pool');

module.exports = {
  getUserAccountById: async (accountId) => {
    let conn;
    let result;

    try {
      conn = await pool.getConnection();

      var sql = "SELECT account_id, account_username, account_image_url FROM user_accounts "
        + "WHERE account_id = ?";

      var rows = await conn.query(sql, [accountId]);

      result = {
        isError: false,
        data: rows
      };
    } catch (error) {
      result = {
        isError: true,
        errorMessage: error.message
      };
    } finally {
      if (conn)
        conn.release();
    }

    return result;
  }
}
```

MariaDB

ดึง connection ใน pool มา 1
ตัวแล้วนำไปเก็บไว้ใน conn

```
const pool = require('../libs/db_pool');

module.exports = {
  getUserAccountById: async (accountId) => {
    let conn;
    let result;

    try {
      conn = await pool.getConnection();

      var sql = "SELECT account_id, account_username, account_image_url FROM user_accounts "
        + "WHERE account_id = ?";

      var rows = await conn.query(sql, [accountId]);

      result = {
        isError: false,
        data: rows
      };
    } catch (error) {
      result = {
        isError: false,
        errorMessage: error.message
      }
    } finally {
      if (conn)
        conn.release();

      return result;
    }
  }
}
```

MariaDB

สร้างคำสั่ง SQL เพื่อส่งไปยัง
ฐานข้อมูล

เครื่องหมาย ? เป็นการกำหนด
ว่าเราจะแทนค่าตัวแปรเข้าไปยัง
ส่วนนั้น

```
const pool = require('../libs/db_pool');

module.exports = {
  getUserAccountById: async (accountId) => {
    let conn;
    let result;

    try {
      conn = await pool.getConnection();

      var sql = "SELECT account_id, account_username, account_image_url FROM user_accounts "
        + "WHERE account_id = ?";

      var rows = await conn.query(sql, [accountId]);

      result = {
        isError: false,
        data: rows
      };
    } catch (error) {
      result = {
        isError: false,
        errorMessage: error.message
      }
    } finally {
      if (conn)
        conn.release();

      return result;
    }
  }
}
```

MariaDB

ส่งคำสั่ง SQL พร้อมแนบตัวแปร

```
const pool = require('../libs/db_pool');

module.exports = {
  getUserAccountById: async (accountId) => {
    let conn;
    let result;

    try {
      conn = await pool.getConnection();

      var sql = "SELECT account_id, account_username, account_image_url FROM user_accounts "
        + "WHERE account_id = ?";

      var rows = await conn.query(sql, [accountId]);

      result = {
        isError: false,
        data: rows
      };
    } catch (error) {
      result = {
        isError: false,
        errorMessage: error.message
      }
    } finally {
      if (conn)
        conn.release();

      return result;
    }
  }
}
```

MariaDB

ใส่ข้อมูลให้กับตัวแปร result

isError: false แปลว่าไม่เกิด
ปัญหา

data: rows นำข้อมูลที่สืบค้นได้
ใส่ในตัวแปรชื่อ data

```
const pool = require('../libs/db_pool');

module.exports = {
  getUserAccountById: async (accountId) => {
    let conn;
    let result;

    try {
      conn = await pool.getConnection();

      var sql = "SELECT account_id, account_username, account_image_url FROM user_accounts "
        + "WHERE account_id = ?";

      var rows = await conn.query(sql, [accountId]);

      result = {
        isError: false,
        data: rows
      };
    } catch (error) {
      result = {
        isError: false,
        errorMessage: error.message
      };
    } finally {
      if (conn)
        conn.release();

      return result;
    }
  }
}
```

MariaDB

ใส่ข้อมูลให้กับตัวแปร result

isError: true แปลว่าเกิด error
ขึ้น

errorMessage:
error.message นำข้อความ
แจ้ง error ใส่ในตัวแปรชื่อ
errorMessage

```
const pool = require('../libs/db_pool');

module.exports = {
  getUserAccountById: async (accountId) => {
    let conn;
    let result;

    try {
      conn = await pool.getConnection();

      var sql = "SELECT account_id, account_username, account_image_url FROM user_accounts "
        + "WHERE account_id = ?";

      var rows = await conn.query(sql, [accountId]);

      result = {
        isError: false,
        data: rows
      };
    } catch (error) {
      result = {
        isError: true,
        errorMessage: error.message
      };
    } finally {
      if (conn) {
        conn.release();
      }
      return result;
    }
  }
}
```

MariaDB

คืน connection ให้กับ pool

```
const pool = require('../libs/db_pool');

module.exports = {
  getUserAccountById: async (accountId) => {
    let conn;
    let result;

    try {
      conn = await pool.getConnection();

      var sql = "SELECT account_id, account_username, account_image_url FROM user_accounts "
        + "WHERE account_id = ?";

      var rows = await conn.query(sql, [accountId]);

      result = {
        isError: false,
        data: rows
      };
    } catch (error) {
      result = {
        isError: false,
        errorMessage: error.message
      };
    } finally {
      if (conn)
        conn.release();
    }

    return result;
  }
}
```

MariaDB

การติดต่อฐานข้อมูลจะเกิดการ
รอสว่างการทำงาน

- Asynchronous

คำสั่ง await จะทำให้
JavaScript รอจนกว่าจะทำงาน
บรรลุตอนนี้เสร็จ แล้วจึงไปทำงาน
คำสั่งต่อไป

เมื่อเราใช้ await เราจะต้องตั้ง
ค่าให้ function ของเราเป็น
async

```
const pool = require('../libs/db_pool');

module.exports = {
  getUserAccountById: async (accountId) => {
    let conn;
    let result;

    try {
      conn = await pool.getConnection();

      var sql = "SELECT account_id, account_username, account_image_url FROM user_accounts "
        + "WHERE account_id = ?";

      var rows = await conn.query(sql, [accountId]);

      result = {
        isError: false,
        data: rows
      };
    } catch (error) {
      result = {
        isError: false,
        errorMessage: error.message
      };
    } finally {
      if (conn)
        conn.release();

      return result;
    }
  }
}
```

MariaDB

แก้ไข server.js

ดึง function จาก
user_account.js มาเก็บในตัว
และ userAccountModel

```
const http = require('http');  
const bp = require('body-parser');  
const express = require('express');  
const userAccountModel = require('./models/user_account');  
  
const app = express();  
app.use(bp.urlencoded({ extended: true }));  
app.use(bp.json());
```

MariaDB

เพิ่ม endpoint สำหรับดึงข้อมูล
ของ user ด้วย account_id

```
app.get("/api/user/:accountId", async (req, res) => {  
  const accountId = req.params.accountId;  
  const response = await userAccountModel.getUserAccountById(accountId);  
  res.send(JSON.stringify(response));  
});  
  
app.listen(port, () => {  
  console.log(`Server running at http://${hostname}:${port}`);  
});
```

MariaDB

ตัวแปร :accountId ใน URI จะถูกส่งมาใน params ของ request

```
app.get("/api/user/:accountId", async (req, res) => {  
  const accountId = req.params.accountId;  
  const response = await userAccountModel.getUserAccountById(accountId);  
  res.send(JSON.stringify(response));  
});  
  
app.listen(port, () => {  
  console.log(`Server running at http://${hostname}:${port}`);  
});
```

MariaDB

เรียกใช้ function ชื่อ
getUserAccountById จาก
user_account.js

```
app.get("/api/user/:accountId", async (req, res) => {  
  const accountId = req.params.accountId;  
  const response = await userAccountModel.getUserAccountById(accountId);  
  res.send(JSON.stringify(response));  
});  
  
app.listen(port, () => {  
  console.log(`Server running at http://${hostname}:${port}`);  
});
```

MariaDB

แก้ไข test.http เพื่อทดสอบ API

```
HTTP/1.1 200 OK
X-Powered-By: Express
Content-Type: text/html; charset=utf-8
Content-Length: 106
ETag: W/"6a-YNsTB+JJ+/34qqTWNuu/hCLxD1c"
Date: Wed, 01 Jul 2026 04:23:13 GMT
Connection: close

{
  "isError": false,
  "data": [
    {
      "account_id": 1,
      "account_username": "rodri66",
      "account_image_url": "rodri66.png"
    }
  ]
}
```

```
###
Send Request
POST http://localhost:3000/api/multiple_by_2 HTTP/1.1
Content-Type: application/json

{
  "no_1": 20,
  "no_2": 50.5
}
```

```
###
Send Request
GET http://localhost:3000/api/user/1 HTTP/1.1
Content-Type: application/json

{

}
```